

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



FIUSAC
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Proyecto 1

Desarrollo, Conexión y Gestión de
Contenedores en Entornos Virtualizados

PONDERACIÓN: 15pts

 **Tiempo estimado: 50hrs**

Índice

1. MARCO FORMATIVO.....	3
1.1. Valor.....	3
1.2. Competencia(s).....	3
1.3. Objetivo SMART.....	3
2. Resumen Ejecutivo.....	4
3. Enunciado del Proyecto.....	4
4.1 Descripción del problema a resolver.....	4
4.1.1 Instrucciones Técnicas:.....	5
EJEMPLO General.....	7
Flujo de Funcionamiento.....	7
Puntos Clave a Recordar.....	8
3.3 Entregables.....	9
4. Material de apoyo.....	9
5. Metodología.....	9
6. Recursos y herramientas a utilizar.....	10
7. Desarrollo de Habilidades Blandas.....	10
8. Cronograma.....	10
9. Rúbrica de Calificación.....	11
9.1 Requisitos para optar a la calificación.....	11
9.2 Resumen de Puntuaciones.....	12
Detalle de la Calificación.....	13

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad	Mejorar y entregar nuestros trabajos como parte de las responsabilidades del curso

1.2. Competencia(s)

Indique la competencia general del curso y la competencia específica trabajada en esta lectura.

Tipo de Competencia	General
Competencia General	Aprender el uso y funciones de un SO
Competencia Específica	Aprender Linux a fondo y las máquinas virtuales para su uso

1.3. Objetivo SMART

Define el/los objetivo(s) que se pretende cumplir con la tarea planteada, con un enfoque SMART, toma en consideración que un objetivo SMART, va enfocado a la habilidad(y no a tanto a la teoría

SMART	Definición	Objetivo redactado
Específico (¿Qué?)	El objetivo es concreto y tangible.	Los estudiantes serán capaces de identificar y configurar correctamente los elementos básicos de una red (topologías, direccionamiento IP, dispositivos de red y protocolos fundamentales).
Medible (¿Cuánto?)	El objetivo tiene una medida objetiva de éxito.	Lograr al menos un 80% de aciertos en las prácticas y evaluaciones.
Alcanzable (¿Cómo?)	El objetivo debe ser posible con los recursos disponibles.	Utilizando únicamente las herramientas de laboratorio disponibles en clase.
Realista (¿Para qué?)	El objetivo contribuye a metas más amplias.	con habilidades prácticas que fortalezcan la formación en informática y sistemas, preparándose para cursos más avanzados en redes y seguridad.
A Tiempo (¿Cuándo?)	El objetivo tiene fecha límite o mejor aún un cronograma de hitos de progreso	Cumplir este aprendizaje en un plazo máximo de 12 semanas.

2. Resumen Ejecutivo

El presente proyecto tiene como objetivo principal el diseño e implementación de un entorno virtualizado que integre el uso de máquinas virtuales (VMs) y contenedores, empleando tecnologías modernas como Docker, Containerd, Go y Zot. Esta arquitectura permite simular entornos reales de desarrollo utilizados en la industria, enfocados en la contenerización, el almacenamiento de imágenes de contenedores, conexión entre API's y la gestión eficiente de recursos.

3. Enunciado del Proyecto

El desarrollo de aplicaciones distribuidas requiere infraestructura moderna que permita portabilidad, seguridad y eficiencia. Este proyecto propone crear un entorno simulado para comprender la contenerización, el almacenamiento de imágenes y la comunicación entre servicios, usando herramientas reales de la industria.

4.1 Descripción del problema a resolver

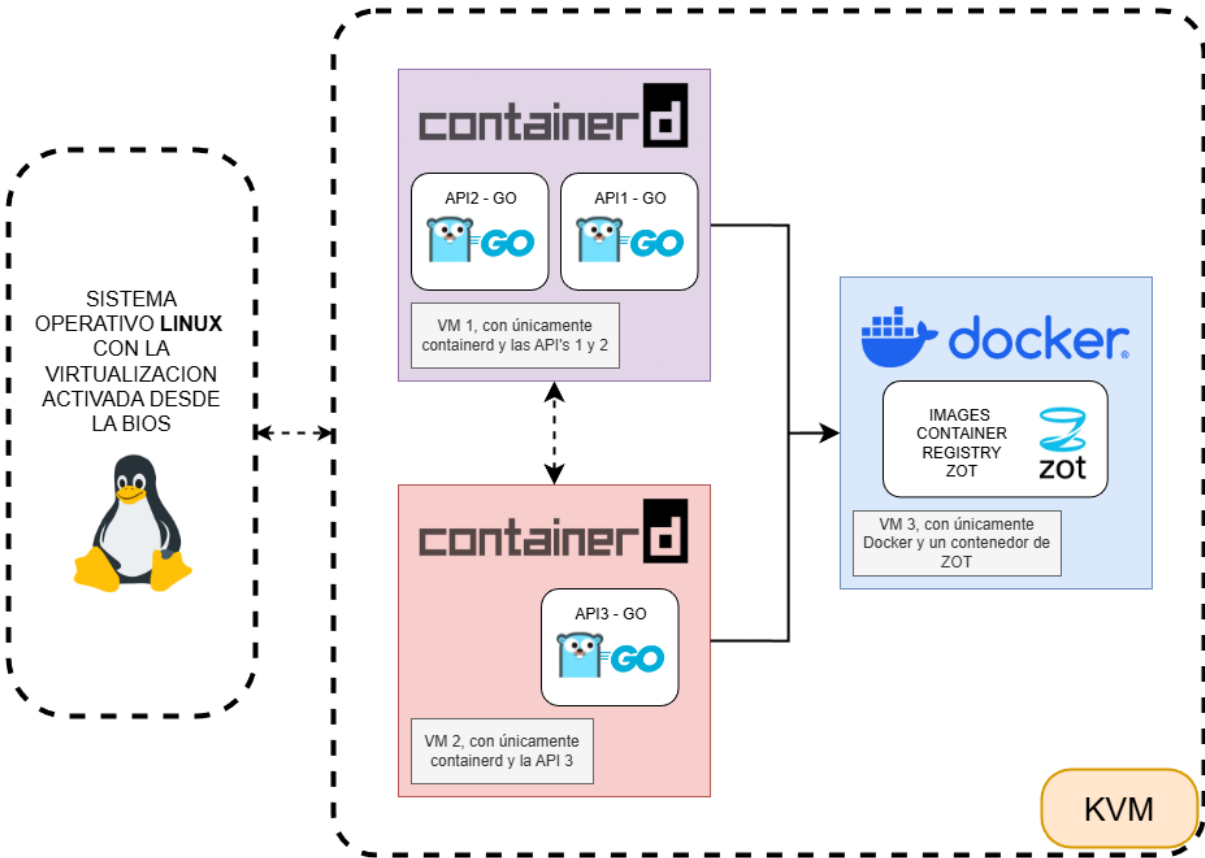
- Instalación de 3 VMs con distintos propósitos (servicios, registros) usando KVM.
- Implementación de APIs REST con contenedores personalizados.
- Uso de registros privados para distribución de imágenes.
- Comunicación cruzada (REST) entre servicios contenerizados.
- Documentación técnica completa.

4.1.1 Instrucciones Técnicas:

Infraestructura:

Máquina Virtual	Runtime para los Contenedores	Contenedor(es) Ejecutado(s)
VM1	Containerd	API1, API2
VM2	Containerd	API3
VM3	Docker	ZOT

Diagrama:



Comunicación entre APIs:

- Protocolo de Comunicación: REST/HTTP
- Formato de Datos: JSON
- Variables Personalizadas: Cada estudiante debe reemplazar **#CARNET** por su número de carnet.

Endpoints que deben realizar:

API	Endpoint Principal	Endpoint de llamada 1	Endpoint de llamada 2
API1	GET /health	GET /api1/#CARNET/call-api2	GET /api1/#CARNET/call-api3
API2	GET /health	GET /api2/#CARNET/call-api1	GET /api2/#CARNET/call-api3
API3	GET /health	GET /api3/#CARNET/call-api1	GET /api3/#CARNET/call-api2

Estructura de Respuesta (JSON)

/health

```
{
  "status": "UP",
  "message": "API# is Ready",
  "timestamp": <DATE>,
  "VM": #vm,
  "carnet": <#CARNET>
}
```

/api#/#CARNET/call-api#

```
{
  "apiname": "API#",
  "message": "The API# located on the VM# is working",
  "connection": true,
  "carnet": <#CARNET>
}
```

EJEMPLO General

En este ejemplo se describe el flujo de comunicación cuando una API (API2) verifica el estado de funcionamiento de otra API (API1) mediante una llamada HTTP interna.

El usuario realiza una solicitud a API2, y esta se encarga de consultar el endpoint de salud (**/health**) de API1 para determinar si se encuentra operativa.

Flujo de Funcionamiento

Solicitud del usuario.

Desde un navegador o terminal, el usuario realiza la siguiente petición:

GET /api2/#CARNET/call-api1

1. Procesamiento en API2

Al recibir la solicitud, API2 ejecuta las siguientes acciones:

- Prepara una llamada HTTP interna. (usando la herramienta de tu elección)
- Intenta contactar el endpoint **/health** de API1.
- Espera la respuesta de API1.

Respuesta de API1:

Si API1 se encuentra funcionando correctamente, responde con el siguiente formato JSON:

```
{
  "status": "UP",
  "message": "API1 is Ready",
  "timestamp": "2029-01-31T10:30:00Z",
  "VM": "#VM",
  "carnet": "#CARNET"
}
```

2. Análisis de la respuesta en API2

- Si el campo **"status"** es igual a **"UP"**, se considera que API1 está operativa.
- Si ocurre un **error** o el valor de **"status"** **no es "UP"**, se determina que API1 presenta **problemas**.

3. Respuesta final al usuario

- Si API1 está funcionando correctamente el endpoint

GET /api2/#CARNET/call-api1 devuelve:

```
{
  "apiname": "API1",
  "message": "The API1 located on the VM1 is working",
  "connection": true,
  "carnet": <#CARNET>
}
```

- Si API1 NO está funcionando el endpoint

GET /api2/#CARNET/call-api1 devuelve:

```
{
  "apiname": "API1",
  "message": "ERROR: The API1 located on the VM1 is not
working",
  "connection": false,
  "carnet": <#CARNET>
}
```

Puntos Clave a Recordar

- Cada API es independiente y debe desarrollarse por separado.
- El endpoint **/health** siempre debe responder con el mismo formato JSON y **"status": "UP"**.
- Los endpoints de llamadas deben realizar peticiones HTTP a otras APIs.
- El carnet del estudiante debe incluirse en todas las rutas y respuestas.
- Se deben manejar errores adecuadamente; si no se puede contactar a otra API, se debe responder con **"connection": false y su respectivo mensaje de ERROR**.
- Es obligatorio respetar exactamente los formatos JSON especificados.

Personalización:

Los nombres de las imágenes de los contenedores deben llevar el nombre:

[API#-#CARNET]

Los tags quedan a criterio del estudiante.

En la documentación se debe incluir toda la configuración de sus máquinas virtuales y APIs desarrolladas para que tenga validez.

3.3 Entregables

1. Código fuente en repositorio privado GitHub nombre
Carnet#_LAB_SO1_1S2026
Deberá añadir a los auxiliares como colaboradores
Usuarios de auxiliares: @roldyoran, @JoseLorezana272, @KINGR0X
(Agregar a los 3 auxiliares)
2. Manual técnico y guía de instalación
3. Capturas de prueba y evidencia funcional

4. Material de apoyo

Link Repositorio del curso

5. Metodología

1. Código fuente en repositorio privado GitHub nombre
Carnet#_LAB_SO1_1S2026
Deberá añadir a los auxiliares como colaboradores
Usuarios de auxiliares: @roldyoran, @JoseLorezana272, @KINGR0X
(Agregar a los 3 auxiliares)
2. Manual técnico y guía de instalación
3. Capturas de prueba y evidencia funcional

6. Recursos y herramientas a utilizar

Listado de materiales que los estudiantes deberán usar o investigar:

- Docker
- Zot
- qemu-kvm
- ContainerD
- Go

7. Desarrollo de Habilidades Blandas

- Gestión de tiempo personal
- Responsabilidad sobre el diseño e implementación
- Resolución autónoma de problemas
- Autoevaluación y reflexión sobre el trabajo realizado

8. Cronograma

El cronograma describe las etapas clave del proyecto, los plazos estimados para cada una, y el proceso de asignación, elaboración y calificación de las tareas. Los estudiantes deberán seguir este plan para asegurar que el proyecto avance de manera organizada y cumpla con los plazos establecidos. Cada fase incluye la asignación de tareas, el tiempo estimado para su elaboración, y el momento de su calificación.

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	29/01/2026	19/02/2026
Elaboración		
Calificación	20/02/2026	21/02/2026

9. Rúbrica de Calificación

9.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Tema	Descripción	Cumple (Si/No)
Cumplimiento de la tecnología establecida	El desarrollo debe haberse realizado en el lenguaje de programación especificado . Uso de herramientas o frameworks requeridos	si
Gestión y entregas del proyecto	Se deben haber cumplido con todas las entregas parciales según el cronograma. Todos los releases o versiones establecidas así como el código completo desarrollado por el estudiante deben estar disponibles en el repositorio de control de versiones en este caso Github	si
Documentación obligatoria	Se debe entregar documentación técnica completa, incluyendo el manual técnico y su guía de instalación de los contenedores y/o imágenes. El Diagrama de arquitectura y/o flujo del sistema debe estar presente en el informe técnico según indique el tutor. (pueden tomar el diagrama de este mismo enunciado)	si
Pruebas y funcionalidad mínima	El sistema debe ser funcional y cumplir con al menos los requisitos mínimos establecidos como lo son el uso de GO y KVM. Debe haber evidencia de pruebas realizadas y reporte de resultados dentro de su documentación.	si
Otros		

9.2 Resumen de Puntuaciones

Área / Sprint	Actividad Evaluada	Puntos Totales	Puntos Obtenidos
Preparación del Entorno	Instalación y configuración del entorno virtualizado usando KVM	10	
	Instalación y prueba de Docker / Containerd	10	
Subtotal		20	
Desarrollo y Contenerización	Desarrollo de APIs en Go	10	
	Creación de Dockerfiles y construcción de imágenes	10	
	Pruebas locales y ejecución funcional	5	
Subtotal		25	
Registros y Comunicación Cruzado	Configuración de Zot	5	
	Subida y descarga de imágenes entre VMs	10	
	Ejecución cruzada y comunicación entre servicios	10	
Subtotal		25	
Validación y Documentación	Pruebas de validación funcional (APIs y registros)	5	
	Manual técnico y evidencia del flujo completo	5	
	Repositorio en GitHub con estructura y guía de instalación	5	
Subtotal		15	
Habilidades Transversales	Respuesta a preguntas relacionadas al proyecto	5	

	Capacidad de modificar y justificar el código durante revisión	5	
	Innovación y originalidad de la solución	5	
Subtotal Habilidades		15	
TOTAL GENERAL		100	

Detalle de la Calificación

	Criterio/Nivel			Nivel de evaluación			
No.	Criterio de evaluación	Punteo máximo		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido	
1	Habilidades	20					
1.1	Instalación y configuración del entorno virtualizado usando KVM	20	Descripción	FUNCIONA	NO FUNCIONA		
			Rango del punteo	12-20/	11 - 0/		
	Sub-Total de Puntos						
2	Conocimientos	15					
2.1	Habilidades Transversales	15	Descripción	Describe y explica las preguntas propuestas.	La preguntas presentadas no obtienen una respuesta correcta		
			Rango del punteo	9-15/	0-8/		
	Sub-Total de Puntos						
	Total						

